

TP sur JML

Objectifs :

- Comprendre les outils fournis par JML ;
- Comprendre l'intérêt d'exprimer les propriétés d'un programme ;
- Utiliser les contrats comme une aide à la mise au point.

Exercice 1 : JML

1. Lire et chercher à comprendre la classe ExempleDate1, la compiler avec javac et l'exécuter avec java.

Listing 1 — Programme manipulant les dates 1 //

Que penser du programme suivant ?

```
public class ExempleDate1 {  
    public static void main (String args [ ] ) {  
        // Construire les dates  
        Date d1 = new Date ( 20 , 5 , 1989 ) ;  
        Date d2 = new Date ( 13 , 2 , 1993 ) ;  
        Date d3 = new Date ( 31 , 6 , 2001 ) ;  
        // Afficher les dates  
        System . out . println ( "d1* " + d1 ) ;  
        System . out . print ln ( + d2 ) ;  
        System . out . print ln ( + d3 ) ;  
    }  
}
```

2. Compiler maintenant l'application en utilisant jmlc. Il suffit de taper :

```
jmlc Date. java ExempleDate1.java
```

3. Exécuter le programme en tapant :

```
jmlrac ExempleDate1
```

Commenter.

4. Engendrer la documentation en utilisant jml doc. Les contrats sont pris en compte !

Remarque : Le compilateur jmlc instrumente les classes listées sur la ligne de commande. Instrumenter signifie ici ajouter des instructions pour vérifier les contrats exprimés dans les commentaires de comportement. Si un contrat n'est pas satisfait, une exception est levée. C'est une exception qui n'a pas à être récupérée. Elle signale une erreur de programmation.

Attention, seules les classes listées sur la ligne de commande sont instrumentées. Les autres ne le sont pas. En argument de jmlc, on peut également donner des répertoires. Dans ce cas, tous les fichiers .java de ces répertoires sont instrumentés.

Exercice 1 : JML

Considérons le programme ci-dessous.

1. Lire le texte de la classe Person et chercher à comprendre.
2. Compiler le programme avec javac (vérifier si un fichier est généré) et l'exécuter avec java
3. Compiler maintenant l'application en utilisant jmlc. Il suffit de taper : `jmlc Person.java`
4. Vérifier le fichier généré par cette opération, quel est son type ?
5. Exécuter le programme en tapant : `jmlrac Person`
6. Commenter votre résultat.
7. Essayer les commandes de test : `jmlunit Person.java` et `jtest Person.java`
8. Engendrer la documentation en utilisant jml doc. Voir si les contrats sont pris en compte.

```
public class Person {
    private String name;
    private int weight;

    /*@ also
    @ ensures \result != null
    @   && (* \result is a displayable
    @   form of this person *);
    @*/
    public String toString() {
        return "Person(\"" + name + "\", "
            + weight + ")";
    }

    public int getWeight() {
        return weight;
    }

    public void addKgs(int kgs) {
        if (kgs >= 0) {
            weight += kgs;
        } else {
            throw new IllegalArgumentException();
        }
    }

    public Person(String n) {
        name = n; weight = 0;
    }

    public static void main(String[] args){
        System.out.println("Bonjour Person");
        Person p = new Person("totot");
        System.out.print(p.toString());
    } }
```

Exercice 3 : OpenJML

Considérons à nouveau le programme de l'exercice 1 que nous copions dans un nouveau répertoire. Exécutons OpenJML avec les différentes options présentées ci-dessous. Nous vérifierons si l'exécution a généré un nouveau fichier ou non.

1. `java -jar ..\openJML\openjml.jar -check ..\Person.java`
2. `java -jar ..\openJML\openjml.jar -esc ..\Person.java`
3. `java -jar ..\openJML\openjml.jar -compile ..\Person.java`
4. `java -jar ..\openJML\openjml.jar -rac ..\Person.java`
5. `java -jar ..\openJML\openjml.jar -racJavaChecks ..\Person.java`
6. `java -jar ..\openJML\openjml.jar -trace ..\Person.java`

Nota: To run OpenJML, be sure that the java command uses a 1.8 JVM and use the following command line. Here \$OPENJML designates the folder in which the openjml.jar and other installation files reside.

`java -jar $OPENJML /openjml.jar <options> <files>`

or

`java -cp $OPENJML /openjml.jar org.jmlspecs.openjml.Main <options> <files>`

Options:

- `-check` : typecheck only (`-command=check`)
- `compile` : typecheck JML but just compile the Java code (`-command=check`)
- `-counterexample`: show a counterexample for failed static checks
- `-rac` : compile runtime assertion checks (`-command=rac`)
- `-racCheckAssumptions` : enables (default on) checking assume statements as if they were asserts
- `-racCompileToJavaAssert` : compile RAC checks using Java asserts
- `-racJavaChecks` : enables (default on) performing JML checking of violated Java features
- `-racShowSource` : includes source location in RAC warning messages
- `-showNotImplemented` warn if feature not implemented
- `-trace` : show a trace for failed static checks

Exercice 4 : Compiler et exécuter le programme suivant issu de la documentation de JML pour voir les différents messages de retour

1. Avec JML
2. Avec OpenJML

```
package org.jmlspecs.samples.jmltutorial;
import org.jmlspecs.models.JMLDouble;

public class SqrtExample {
    public final static double eps = 0.0001;
    //@ requires x >= 0.0;
    //@ ensures JMLDouble.approximatelyEqualTo(x, \result * \result, eps);
    public static double sqrt(double x) {
        return Math.sqrt(x);
    }
}
```